

UOD-DSM Implementation Report

Unified Object Detector with Dynamic Switch Mechanism

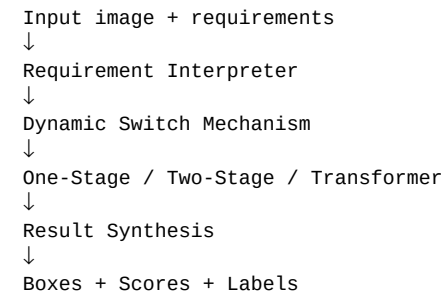
Item	Implementation
Framework	Python + PyTorch + Torchvision
Backbone	ResNet-18 feature extractor
Stage I	Requirement Interpreter
Stage II	Rule-based Dynamic Switch Mechanism
Stage III	One-stage, two-stage and transformer heads
Stage IV	Result Synthesis Module
Data	Synthetic multi-domain object-detection data
Experiments	Baseline vs modified parameters

This implementation converts the UOD-DSM architecture and dynamic-switching methodology described in the research paper into an executable PyTorch pipeline. Selected parameters are then modified and the resulting behavior is compared with the baseline configuration.

1. Concept and Objective

UOD-DSM avoids using one fixed object-detection paradigm for every situation. It dynamically selects a detection paradigm according to operational requirements and scene characteristics.

Main mapping: one-stage detection is speed-oriented, two-stage detection is precision-oriented, and transformer detection is context/complexity-oriented.



The implementation follows the four-stage structure described in the paper: Stage I interprets requirements, Stage II selects the paradigm, Stage III executes the selected detector, and Stage IV produces standardized output.

2. Code Structure

Component	Role
DomainConfig	Stores latency, precision and complexity requirements
MultiDomainDetectionDataset	Generates synthetic images, boxes, labels and metadata
SharedBackbone	Extracts shared visual features using ResNet-18
OneStageHead	Speed-oriented detector head
TwoStageHead	Proposal and refinement detector head
TransformerHead	Query-based context detector head
RequirementInterpreter	Processes metadata and visual features
DynamicSwitchMechanism	Selects the detection paradigm
ResultSynthesisModule	Filters and standardizes final detections
UnifiedObjectDetector_DSM	Combines the complete pipeline

3. Domain Configuration and Synthetic Dataset

The implementation uses a synthetic multi-domain dataset. Each sample contains an image, bounding boxes, class labels, a domain identifier and a three-value metadata vector.

Domain	Latency	Precision	Complexity	Expected routing
Autonomous / Real-Time	15 ms	0.50	0.30	One-stage
Medical Imaging	200 ms	0.95	0.50	Two-stage
Remote Sensing / Aerial	100 ms	0.70	0.90	Transformer

The metadata is represented as [latency budget, required precision, scene complexity]. It is generated inside the implementation, so there is no separate external dataset file for this experiment.

4. Shared Feature Extraction

SharedBackbone uses ResNet-18 from Torchvision. The classification portion is removed and early/intermediate residual layers produce a feature map shared by all detector branches.

```
resnet = torchvision.models.resnet18(weights=None)
self.stem = ...
self.layer1 = resnet.layer1
self.layer2 = resnet.layer2
self.layer3 = resnet.layer3
```

5. Stage III — Detection Heads

The three branches share backbone features but use different detection mechanisms.

Branch	Mechanism	Role
One-stage	Direct dense prediction	Speed
Two-stage	Region proposals + RoI refinement	Precision
Transformer	Object queries + attention	Context / complexity

The one-stage branch directly predicts classes and boxes. The two-stage branch creates candidate proposals, applies RoI processing and refines predictions. The transformer branch uses learnable object queries and transformer decoder layers.

6. Stage I — Requirement Interpreter

RequirementInterpreter processes latency, precision and scene-complexity metadata using an MLP and can combine it with pooled visual features. The representations are fused to produce routing logits.

Metadata + Visual features → Fusion → 3 routing logits

The main experiment uses rule_based DSM routing, so the final decision is made from the configured metadata thresholds rather than the learned routing logits.

7. Stage II — Dynamic Switch Mechanism

The DSM checks the requirements and selects a detector paradigm. The simplified rule structure is:

```
if latency < latency_threshold: one_stage
elif precision > precision_threshold: two_stage
elif complexity > complexity_threshold: transformer
```

Changing these thresholds changes the conditions under which each detector can be selected.

8. Stage III — Activating the Selected Detector

The three detector heads are stored in a dictionary. The selected paradigm determines which head is executed. The evaluation uses batch size one, so each evaluated sample is routed independently.

9. Stage IV — Result Synthesis

ResultSynthesisModule removes detections below the configured confidence threshold, sorts remaining scores and limits the final output count. The standardized outputs are boxes, confidence scores and class labels.

10. UnifiedObjectDetector_DSM

The complete forward flow is:

```
features = backbone(images)
routing_logits = requirement_interpreter(metadata, features)
decisions, paradigm_names = dsm(routing_logits, metadata)
raw_detections = selected_head(features)
final_output = result_synthesis(raw_detections)
```

The model records L_I, L_II, L_III and L_IV. L_total is calculated as L_I + L_II + L_III + L_IV, with backbone timing reported separately.

11. Baseline and Modified Parameters

The architecture is unchanged between experiments. Only selected routing and detector parameters are modified.

Parameter	Baseline	Modified	Effect
Latency threshold	0.30	0.40	Changes speed-priority routing
Precision threshold	0.80	0.75	Changes precision-priority routing
Complexity threshold	0.70	0.60	Changes transformer routing
Confidence threshold	0.10	0.30	Stricter output filtering
Two-stage proposals	32	64	More proposal candidates
Transformer queries	50	100	More object-query slots

Baseline: latency 0.30, precision 0.80, complexity 0.70, confidence 0.10, proposals 32, queries 50.

Modified: latency 0.40, precision 0.75, complexity 0.60, confidence 0.30, proposals 64, queries 100.

12. Evaluation

Metric	Meaning
Average latency	Runtime cost of the adaptive pipeline
L_I / L_II / L_III / L_IV	Time spent in the four processing stages
Paradigm distribution	How often each detector is selected
Average detections	Retained detections per image
Average confidence	Mean confidence of retained detections

The program also benchmarks static detector paradigms. The current implementation uses average confidence as a proxy statistic and does not calculate ground-truth IoU, precision, recall or mAP.

13. End-to-End Execution Flow

1. Load libraries and select CPU/GPU.
2. Define domain requirements.
3. Generate synthetic domain data.
4. Extract shared features with ResNet-18.
5. Interpret metadata and visual features.
6. Apply DSM routing.
7. Execute the selected detection head.
8. Synthesize and filter the output.
9. Record latency and detection statistics.
10. Run baseline.
11. Run modified configuration.
12. Compare baseline and modified behavior.

14. Limitations

The following limitations apply to the current implementation and define how the experimental results should be interpreted.

Area	Limitation
Dataset	Uses synthetic multi-domain data rather than established real-world object-detection datasets. Results therefore demonstrate system behavior rather than real-world detection performance.
Training	No full end-to-end training experiment is performed for the complete heterogeneous detector system.
Detector branches	The one-stage, two-stage and transformer heads are lightweight custom implementations representing the three paradigms; they are not exact official YOLO, Faster R-CNN or DETR configurations.
Accuracy evaluation	No ground-truth IoU, AP or mAP is calculated. Average confidence is only a proxy statistic and must not be described as detection accuracy.
Routing	The reported experiment uses rule-based DSM thresholds. Learned routing is represented in the architecture but is not the primary routing method used for the comparison.
Thresholds	Baseline and modified threshold values are experiment settings for studying parameter sensitivity; they are not claimed to be optimal.
Batch routing	The implementation uses a dominant paradigm for a batch. Batch size one is used during evaluation, so mixed-batch routing is not demonstrated.
Latency	Latency depends on hardware, software environment, batch size and runtime conditions; values are meaningful mainly under the same test setup.
Generalization	Only a small number of synthetic domains and limited class configurations are evaluated, so generalization to other domains and real deployment conditions is not established.
Benchmarking	The experiment does not provide a full large-scale comparison against real-world benchmark detectors. Such evaluation would be needed before making claims about practical superiority.
Reproducibility	Synthetic generation and neural components can introduce variation unless random seeds and execution settings are controlled.

15. Conclusion

The implementation converts the UOD-DSM research concept into an executable four-stage PyTorch pipeline with dynamic selection among one-stage, two-stage and transformer detection paradigms.

Using the same architecture for both experiments allows selected threshold and detector parameters to be changed and their effects to be compared. The experiment demonstrates the dynamic-switching concept and parameter sensitivity, while the limitations above clarify the scope of the current results.

Reference paper: O. Hmidani and E. M. Ismaili Alaoui, "Unified Object Detector With Dynamic Switch Mechanism Based on Deep Learning Advances: A Paradigm-Aware Architecture for Adaptive Detection," IEEE Access, 2026, DOI: 10.1109/ACCESS.2026.3680430.

Implementation source: UOD_DSM_Implementation_Baseline_Modified_NoComments_Fixed.ipynb